

中山大學

SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab2-基于 MPI 的并行矩阵乘法 (进阶)

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2025 年 4 月 2 日

1 实验目的

- 掌握 MPI 集合通信在并行矩阵乘法中的应用
- 学习使用 MPI_Type_create_struct 创建派生数据类型
- 分析不同通信方式和任务划分对并行性能的影响
- 研究并行程序的扩展性和性能优化方法

2 实验内容

- 使用 MPI 集合通信实现并行矩阵乘法
- 使用 MPI_Type_create_struct 聚合进程内变量后通信
- 选做采用：Cyclic 循环划分策略
  - 本次选做采用“循环划分（cyclic）”策略。具体做法是将矩阵 A 的行按行号 r 分配给进程 $r \bmod p$，使每个进程负责若干不连续的行；矩阵 B 则通过广播（`Bcast`）分发给所有进程。在实现上，根进程将属于同一进程的行打包成连续缓冲区，再通过点对点发送或在需要时使用 `Scatterv` 将打包后的块发送到对应进程；各进程收到本地行后与 B 进行本地矩阵乘法，得到局部 C，最后按相同打包格式将局部结果发送回根进程并重排到最终矩阵。
  - 优点：循环划分能在行负载不均的情况下提供更细粒度的负载均衡，避免某些进程成为瓶颈。
  - 缺点：需要行打包/重排，降低数据连续性和缓存局部性，增加额外通信开销；在数据和计算均匀时，收益较小。

3 实验结果

3.1 性能分析

根据运行结果，填入下表以记录不同线程数量和矩阵规模下的运行时间：

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.004960	0.022325	0.436042	3.530934	48.388498
2	0.003309	0.012668	0.210489	1.783088	27.215055
4	0.001855	0.007965	0.112387	0.892029	18.724228
9	0.001093	0.004766	0.072903	0.470796	8.971761
16	0.000652	0.003644	0.041478	0.313412	4.847848

表 1: 用 MPI 集合通信实现

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.004906	0.021852	0.433210	3.520893	50.063990
2	0.003094	0.012806	0.227250	1.781493	28.013393
4	0.001807	0.009149	0.129595	0.873602	13.417061
9	0.000853	0.005054	0.067090	0.494440	6.133437
16	0.000644	0.002910	0.048008	0.317767	4.031495

表 2: 用 MPI_Type_create_struct 聚合进程内变量后通信

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.005200	0.022018	0.438774	3.525712	48.730933
2	0.003242	0.013055	0.218182	1.780729	27.169532
4	0.001600	0.009132	0.108757	0.895208	18.657597
9	0.001072	0.005527	0.071857	0.477825	9.180023
16	0.000603	0.002863	0.042574	0.305457	4.360419

表 3: Cyclic 循环划分策略

4 实验分析

根据运行时间，结合实现代码对程序并行性能及扩展性进行分析。

4.1 方法实现要点

- 集合通信实现（`coll`）：根进程用 `Scatterv` 将 A 的行块分发，各进程用 `Bcast` 接收 B，计算后用 `Gatherv` 汇总 C。
- 点对点实现（`p2p`）及元数据聚合：根进程用 `MPI_Send` 与 `MPI_Recv` 分发 A 的块和 B，使用 `MPI_Type_create_struct` 聚合每个进程的元数据（行数与偏移）并广播以简化进程间协商。
- 块划分（`block`）：根进程将 A 的连续行块分配给各进程，适合数据和计算均匀的情况，通信模式较简单。
- 循环划分（`cyclic`）：根进程将属于同一进程的非连续行打包后发送，各进程计算局部结果并按相同规则重排回 C。

4.2 定量实验结果

4.2.1 coll vs. p2p

表 4: coll 与 p2p（block 划分）对比：t_1, t_{16}, 加速比与效率

矩阵规模	方法	t_1(s)	t_{16}(s)	加速比	效率
128	coll	0.004960	0.000652	7.607	47.55%
128	p2p	0.004906	0.000644	7.618	47.61%
512	coll	0.436042	0.041478	10.513	65.70%
512	p2p	0.433210	0.048008	9.024	56.40%
2048	coll	48.388498	4.847848	9.981	62.38%
2048	p2p	50.063990	4.031495	12.418	77.61%

4.2.2 block vs. cyclic

表 5: block 与 cyclic（在 coll 下）对比：t_1, t_{16}, 加速比与效率

矩阵规模	划分方式	t_1(s)	t_{16}(s)	加速比	效率
128	block	0.004960	0.000652	7.607	47.55%
128	cyclic	0.005200	0.000603	8.624	53.90%
512	block	0.436042	0.041478	10.513	65.70%
512	cyclic	0.438774	0.042574	10.306	64.41%
2048	block	48.388498	4.847848	9.981	62.38%
2048	cyclic	48.730933	4.360419	11.176	69.85%

4.3 实验结果分析

测量方法与指标 程序在每个配置下进行了多次运行并对单个进程的运行时间取平均，最终以各进程平均时间的最大值作为该配置的计时结果。记单进程时间为 t_1，p 个进程时的时间为 t_p，加速比 $S = t_1/t_p$，并行效率 $E = S/p$。

总体可扩展性 随着进程数增加，运行时间总体下降，说明计算部分被有效并行化；但加速比均低于理想的线性增长（$S < p$），效率随着问题规模和通信开销的相对大小而变化：对于较小问题（例如矩阵规模 128），通信和同步开销占比高，效率较低（表中 $E \approx 47\%$）；对于中等规模（512），效率上升（例如 **coll** 在 512 时 $S \approx 10.5$, $E \approx 65.7\%$），说明计算占比增加，能够更好地被并行化；对于很大的问题（2048），效率仍然没有达到 100%，但在某些实现下（**p2p**）效率可达 77.6%，表明计算—通信比、MPI 实现和具体传输策略共同决定了最终扩展性。

coll（集合通信）与 p2p（点对点）比较 小 / 中规模（如 512）：**coll** 的 t_{16}（0.041478 s）优于 **p2p**（0.048008 s），说明在这类消息尺寸下，**MPI_Scatterv**/**MPI_Bcast**/**MPI_Gatherv** 的实现和优化（如树形广播、管道化）能有效降低通信成本。大规模（如 2048）：**p2p** 在本次实验中得到更好的 t_{16}（4.031495 s，$S \approx 12.42$，$E \approx 77.6\%$），这可能由实验环境与 MPI 实现细节导致：点对点发送/接收在大消息和高带宽网络上有时能避免集合通信内部的额外复制或同步延迟；另外，**p2p** 中 root 端对数据的直接分发和收集在本机内存拷贝路径上可能比集合通信的某些中间缓冲实现更优。总体来看，两种方式的优劣与消息大小、网络/实现特性、以及根进程的打包开销有关。

block（块划分）与 cyclic（循环划分）比较 **block** 优点是数据连续、缓存友好、打包开销小；**cyclic** 能在存在不均匀负载或系统噪声时提高负载均衡。实验数据表明：**cyclic** 在并行度较高且矩阵较大时能获得更好加速比（例如在 2048 串行下 **cyclic** 的 $S \approx 11.18$，优于 **block** 的 9.98），说明循环划分减少了某些进程成为瓶颈的情况，提升了总体并行利用率；同时 **cyclic** 会引入额外的行打包/重排开销，导致单进程时间略有增加，但在多进程环境下换来更高的并行效率，符合划分粒度权衡的理论实际。

与理论（Amdahl / Gustafson）的一致性 理论上，理想线性加速需要极小的串行部分和零通信开销；实际系统中通信、同步、打包/拷贝和 MPI 内部实现导致串行化开销 $s > 0$，根据 Amdahl 定理 $S \leq 1/(s + (1-s)/p)$，因此观测到 $S < p$ 是符合预期的。从 Gustafson 视角（随问题规模放大并行部分占比增大），当矩阵规模上升时加速比和效率趋于提高（如从 128 到 512 再到 2048 的趋势），也与理论预期一致：更大的计算量掩盖了固定的通信开销，从而获得更好的扩展性。

探究性能优化方法

- 使用非阻塞通信（**MPI_Isend/MPI_Irecv + MPI_Waitall**）或重叠通信与计算以隐藏通信延迟；
- 对循环划分使用 MPI 派生类型（如 **MPI_Type_vector/MPI_Type_create_subarray**）避免手动打包，从而减少内存拷贝；
- 在本地使用缓存友好的块状矩阵乘法或调用已优化的 BLAS（OpenBLAS/Intel MKL）提高单节点计算性能，进而改善通信—计算比；

总体而言，并行化带来显著加速，但受通信和实现细节约束；进一步按照建议优化方法进行优化后能进一步接近理论上更高的并行效率。